



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 3
по курсу «Компьютерные сети»
«Протокол одноранговой сети»

Студент группы ИУ9-32Б Ивенкова О. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Задание

Краткое описание варианта одноранговой сетевой службы приведено далее.
Чат (кольцо)

Топология: кольцевой список.

Информация, известная пиру при запуске: его IP-адрес и порт, а также IP-адрес и порт следующего пира в кольцевом списке (следующий пир не обязан быть заранее запущен).

Описание службы: каждый пир имеет графический пользовательский интерфейс (или web-интерфейс), через который осуществляется просмотр и передача сообщений в чате.

Основные требования к сетевой службе:

1. в качестве формата сообщений для протокола взаимодействия пиров нужно использовать JSON;
2. полная проверка данных, получаемых из сети;
3. устойчивость к обрыву соединения;
4. ведение подробного лога всех ошибок, а также других важных событий (установка и завершение соединения с соседним пиром, приём и передача сообщений, и т.п.).

2 Результаты

Исходный код программы представлен в листинге 1–3.

Листинг 1: Реализация файла peer1.go

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6     "log"
7     "net"
8     "time"
9
10    "net/http"
11    "github.com/gorilla/websocket"
12 )
```

```

13
14 type Message struct {
15     Sender    string 'json:"sender"'
16     Receiver  string 'json:"receiver"'
17     Message   string 'json:"message"'
18     Timestamp string 'json:"timestamp"'
19 }
20
21 var (
22     myIP = "185.102.139.161"
23     myPort = "55420"
24     nextPeerIP = "185.102.139.168"
25     nextPeerPort = "55421"
26 )
27
28 func main() {
29     log.Printf("Starting peer at %s:%s, next peer: %s:%s\n", myIP,
30         myPort, nextPeerIP, nextPeerPort)
31
32     // Start the server to receive messages
33     go startPOSTServer()
34     go startServer(myIP, myPort)
35     go startWebSocketServer()
36
37     // The loop for sending messages is placed in a separate goroutine
38     go handleUserInput()
39     // The main goroutine will wait for other goroutines to finish (or
40     // will be used for monitoring the state)
41     select {} // This blocks the main thread and allows goroutines to
42     continue working
43 }
44
45 func handleUserInput() {
46     for {
47         time.Sleep(5 * time.Second) // Delay before the next attempt
48         var input string
49         fmt.Printf("Enter a message: ")
50         fmt.Scanln(&input)
51
52         // Send a message to the next peer
53         message := Message{
54             Sender:    myIP + ":" + myPort,
55             Receiver:  nextPeerIP + ":" + nextPeerPort,
56             Message:   input,
57             Timestamp: time.Now().Format(time.RFC3339),
58         }
59     }
60 }

```

```

56
57     // Check peer availability before sending
58     if isPeerAvailable(nextPeerIP, nextPeerPort) {
59         err := sendMessage(nextPeerIP + ":" + nextPeerPort, message)
60         if err != nil {
61             log.Println("Error sending message:", err)
62         }
63     } else {
64         log.Println("Next peer is unavailable. Trying again later.")
65     }
66 }
67 }
68
69 // Function to start the server to receive messages
70 func startServer(ip string, port string) {
71     listener, err := net.Listen("tcp", ip+": "+port)
72     if err != nil {
73         log.Fatal("Error starting server:", err)
74     }
75     defer listener.Close()
76
77     log.Println("Server started at", ip+": "+port)
78     updatePeerState(ip + ":" + port, true)
79
80     for {
81         conn, err := listener.Accept()
82         if err != nil {
83             log.Println("Error accepting client connection:", err)
84             continue
85         }
86
87         go handleConnection(conn)
88     }
89 }
90
91 // Handling incoming connections
92 func handleConnection(conn net.Conn) {
93     defer conn.Close()
94
95     buffer := make([]byte, 4096)
96     n, err := conn.Read(buffer)
97     if err != nil {
98         //log.Println("Error reading data:", err)
99         return
100     }
101

```

```

102     var message Message
103     err = json.Unmarshal(buffer[:n], &message)
104     if err != nil {
105         //log.Println("Error parsing message:", err)
106         return
107     }
108     updatePeerState(message.Sender, true)
109
110     log.Printf("Received message from %s: %s\n", message.Sender, message
111     .Message)
112     sendNewMessage(message)
113     fmt.Printf("Enter a message: ")
114 }
115 // Send a message to the next peer
116 func sendMessage(nextAddress string, message Message) error {
117     conn, err := net.Dial("tcp", nextAddress)
118     if err != nil {
119         log.Println("Error connecting to the next peer:", err)
120         return err
121     }
122     defer conn.Close()
123
124     jsonMessage, _ := json.Marshal(message)
125     _, err = conn.Write(jsonMessage)
126     if err != nil {
127         log.Println("Error sending message:", err)
128         return err
129     }
130
131     log.Println("Message sent:", message)
132     sendNewMessage(message)
133     return nil
134 }
135
136 // Check peer availability using ping
137 func isPeerAvailable(peerIP, peerPort string) bool {
138     var address string
139     if peerIP == "localhost" {
140         // Check the local port
141         address = "localhost:" + peerPort
142     } else {
143         // Check external IP and port
144         address = peerIP + ":" + peerPort
145     }
146

```

```

147 // Use net.DialTimeout to check peer availability with a timeout
148 conn, err := net.DialTimeout("tcp", address, 5*time.Second)
149 if err != nil {
150     log.Printf("Peer %s is unavailable: %v", address, err)
151     //notifyPeerStatus(peerIP, "disconnected")
152     return false
153 }
154 defer conn.Close()
155 //notifyPeerStatus(peerIP, "connected")
156 //log.Printf("Peer %s is available", address)
157 updatePeerState(peerIP + ":" + peerPort, true)
158 return true
159 }
160
161 /////////////////////////////////////////////////// WebSocket
162 ///////////////////////////////////////////////////
163 type PeerState struct {
164     IP          string 'json:"ip"'
165     Port        string 'json:"port"'
166     Connected    bool   'json:"connected"'
167 }
168
169 type DashboardMessage struct {
170     EventType string 'json:"event_type"' // For example, "
171     state_update" or "new_message"
172     PeerState PeerState 'json:"peer_state,omitempty"'
173     Message    Message    'json:"message,omitempty"'
174 }
175
176 var peers = map[string]PeerState{
177     "185.102.139.161:55420": {IP: "185.102.139.161", Port: "55420",
178     Connected: false},
179     "185.102.139.168:55421": {IP: "185.102.139.168", Port: "55421",
180     Connected: false},
181     "185.102.139.169:55422": {IP: "185.102.139.169", Port: "55422",
182     Connected: false},
183 }
184
185 var clients = make(map[*websocket.Conn] bool)
186 var broadcast = make(chan DashboardMessage)
187
188 var upgrader = websocket.Upgrader{
189     CheckOrigin: func(r *http.Request) bool {
190         return true
191     },
192 }
193 }

```

```

188
189 // Start the WebSocket server on each peer
190 func startWebSocketServer() {
191     //http.HandleFunc("/", handleRoot) // Handler for the root route
192     http.HandleFunc("/ws", handleConnections)
193     go handleMessages()
194     log.Printf("WebSocket started on 55430\n")
195     err := http.ListenAndServe(":55430", nil)
196     if err != nil {
197         log.Fatalf("Failed to start WebSocket: %v", err)
198     }
199 }
200
201 // Handle new WebSocket connections
202 func handleConnections(w http.ResponseWriter, r *http.Request) {
203     ws, err := upgrader.Upgrade(w, r, nil)
204     if err != nil {
205         log.Println("Error connecting to WebSocket:", err)
206         return
207     }
208     defer ws.Close()
209
210     clients[ws] = true
211
212     // Initial send of all peer states
213     for _, peer := range peers {
214         message := DashboardMessage{
215             EventType: "state_update",
216             PeerState: peer,
217         }
218         ws.WriteJSON(message)
219     }
220
221     // Wait for connection termination
222     for {
223         _, _, err := ws.ReadMessage()
224         if err != nil {
225             log.Println("WebSocket client disconnected:", err)
226             delete(clients, ws)
227             break
228         }
229     }
230 }
231
232 // Send messages to all connected WebSocket clients
233 func handleMessages() {

```

```

234     for {
235         msg := <-broadcast
236         for client := range clients {
237             err := client.WriteJSON(msg)
238             if err != nil {
239                 log.Println("Error sending message to WebSocket client:"
240 , err)
241                 client.Close()
242                 delete(clients, client)
243             }
244         }
245     }
246
247 // Function to update peer states
248 func updatePeerState(address string, connected bool) {
249     peer := peers[address]
250     peer.Connected = connected
251     peers[address] = peer
252
253     broadcast <- DashboardMessage{
254         EventType: "state_update",
255         PeerState: peer,
256     }
257 }
258
259 // Example of sending a new message
260 func sendNewMessage(message Message) {
261     broadcast <- DashboardMessage{
262         EventType: "new_message",
263         Message:    message,
264     }
265 }
266
267 //////////////////////////////////////// POST
268 ////////////////////////////////////////
269 func startPOSTServer() {
270     http.HandleFunc("/", serveHTML)
271     http.HandleFunc("/send", handlePostRequest)
272     log.Printf("HTTP server started at http://185.102.139.161:55440")
273     log.Fatal(http.ListenAndServe(":55440", nil))
274 }
275
276 func serveHTML(w http.ResponseWriter, r *http.Request) {
277     http.ServeFile(w, r, "index.html") // Assuming index.html is in the
278     same directory

```

```

277 }
278
279 // Handling POST requests
280 func handlePostRequest(w http.ResponseWriter, r *http.Request) {
281     var reqBody struct {
282         Message string `json:"message"`
283     }
284     err := json.NewDecoder(r.Body).Decode(&reqBody)
285     if err != nil {
286         http.Error(w, "Invalid request body", http.StatusBadRequest)
287         return
288     }
289
290     // Create a message for the next peer
291     message := Message{
292         Sender:    myIP + ":" + myPort,
293         Receiver:  nextPeerIP + ":" + nextPeerPort,
294         Message:   reqBody.Message,
295         Timestamp: time.Now().Format(time.RFC3339),
296     }
297
298     // Send the message to the next peer
299     err = sendMessage(nextPeerIP + ":" + nextPeerPort, message)
300     if err != nil {
301         http.Error(w, "Error sending message", http.
302             StatusInternalServerError)
303         return
304     }
305
306     w.Write([]byte("Message sent successfully"))
307 }

```

Листинг 2: Реализация файла index.html

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0"
6     >
7     <title>Message Sender</title>
8 </head>
9 <body>
10     <h1>Send Message to Peer</h1>
11     <form id="messageForm">
12         <label for="sender">Sender Peer (IP:Port):</label>

```

```

12      <input type="text" id="sender" name="sender" placeholder="
13      185.102.139.161:55440" required<br>
14      <label for="message">Message:</label>
15      <input type="text" id="message" name="message" placeholder="
16      Enter your message" required<br>
17      <button type="submit">Send Message</button>
18  </form>
19
20  <div id="response"></div>
21
22  <script>
23      const form = document.getElementById("messageForm");
24      form.addEventListener("submit", function(event) {
25          event.preventDefault();
26
27          const sender = document.getElementById("sender").value;
28          const message = document.getElementById("message").value;
29
30          fetch('http://${sender}/send', {
31              method: "POST",
32              headers: {
33                  "Content-Type": "application/json",
34              },
35              body: JSON.stringify({ message: message }),
36          })
37              .then(response => response.text())
38              .then(data => {
39                  document.getElementById("response").innerText = "Message
40                  sent: " + data;
41              })
42              .catch((error) => {
43                  document.getElementById("response").innerText = "Error:
44                  " + error;
45              });
46      });
47  </script>
48 </body>
49 </html>

```

Листинг 3: Реализация файла dashboard.html

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">

```

```

5  <meta name="viewport" content="width=device-width, initial-scale=1.0
">
6  <title>Peer Dashboard</title>
7  <style>
8      body {
9          font-family: Arial, sans-serif;
10     }
11     .peer {
12         margin: 10px 0;
13     }
14     .connected {
15         color: green;
16     }
17     .disconnected {
18         color: red;
19     }
20 </style>
21 </head>
22 <body>
23     <h1>Peer Status Dashboard</h1>
24     <div id="peers">
25         <!-- Peer states will be displayed here -->
26     </div>
27     <h2>Messages</h2>
28     <div id="messages">
29         <!-- New messages will appear here -->
30     </div>
31
32     <script>
33         const peersDiv = document.getElementById("peers");
34         const messagesDiv = document.getElementById("messages");
35
36         const ws = new WebSocket("ws://185.102.139.161:55430/ws");
37
38         ws.onmessage = function(event) {
39             const data = JSON.parse(event.data);
40
41             if (data.event_type === "state_update") {
42                 updatePeerStatus(data.peer_state);
43             } else if (data.event_type === "new_message") {
44                 displayMessage(data.message);
45             }
46         };
47
48         function updatePeerStatus(peer) {

```

```

49         let peerDiv = document.getElementById(peer.ip + ":" + peer.
port);
50
51         if (!peerDiv) {
52             peerDiv = document.createElement("div");
53             peerDiv.id = peer.ip + ":" + peer.port;
54             peerDiv.classList.add("peer");
55             peersDiv.appendChild(peerDiv);
56         }
57
58         peerDiv.textContent = `Peer ${peer.ip}:${peer.port} is ${
peer.connected ? "connected" : "disconnected"}`;
59         peerDiv.className = `peer ${peer.connected ? "connected" : "
disconnected"}`;
60     }
61
62     function displayMessage(message) {
63         const messageDiv = document.createElement("div");
64         messageDiv.textContent = `${message.timestamp} - ${message.
sender} to ${message.receiver}: ${message.message}`;
65         messagesDiv.appendChild(messageDiv);
66     }
67 </script>
68 </body>
69 </html>

```

Результат запуска представлен на рисунках 1–3.

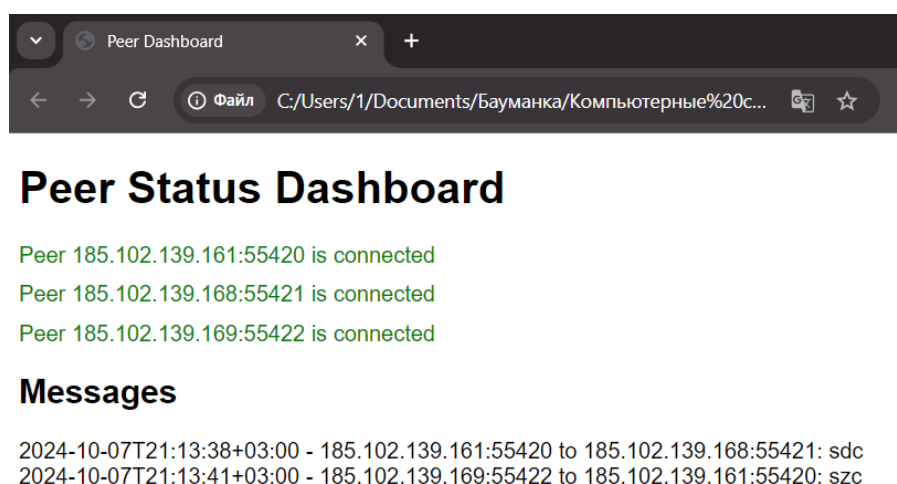
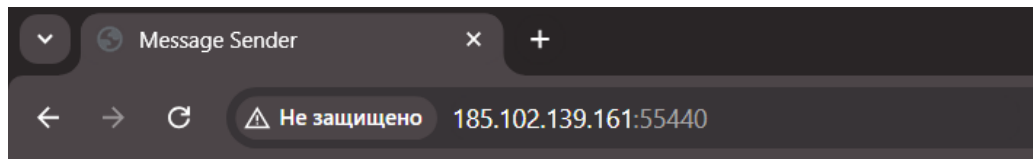


Рис. 1 — Результат со стороны Websocket страницы



Send Message to Peer

Sender Peer (IP:Port):

Message:

Message sent: Message successfully sent to next peer

Рис. 2 — Результат со стороны формы для отправки POST запросов

```
root@net2:~/ivenkova/lab3# root@net2:~/ivenkova/lab3# go run peer1.go
2024/10/07 21:13:02 Запуск пира на 185.102.139.161:55420, следующий пир: 185.102.139.168:55421
2024/10/07 21:13:02 HTTP сервер запущен на http://185.102.139.161:55440
2024/10/07 21:13:02 Сервер запущен на 185.102.139.161:55420
2024/10/07 21:13:02 Запущен веб-сокеты на 55430
Введите сообщение: sdc
2024/10/07 21:13:38 Сообщение отправлено: {185.102.139.161:55420 185.102.139.168:55421 sdc 2024-10-07T21:13:38+03:00}
2024/10/07 21:13:41 Получено сообщение от 185.102.139.169:55422: szc
Введите сообщение: Введите сообщение: 2024/10/07 21:16:04 Получено сообщение от 185.102.139.169:55422: \ax
Введите сообщение: zsc
2024/10/07 21:16:06 Сообщение отправлено: {185.102.139.161:55420 185.102.139.168:55421 zsc 2024-10-07T21:16:06+03:00}
Введите сообщение: 2024/10/07 21:16:48 Сообщение отправлено: {185.102.139.161:55420 185.102.139.168:55421 ascxdzx 2024-10-07T21:16:48+03:00}
```

Рис. 3 — Результат со стороны терминала